Spring MVC

Session 3

REST, JSON & Everything Around the Controller

> Still Spring Core — no Spring Boot. New dependency this session: `jackson-databind`, for JSON conversion.

Quick Recap of Sessions 1 & 2

- `DispatcherServlet` → `HandlerMapping` → `HandlerAdapter` → Controller → `ViewResolver` → `View` — the pipeline never changes
- `@RequestParam`, `@PathVariable`, `@ModelAttribute`, `@Valid` / `BindingResult`, `@ExceptionHandler` / `@ControllerAdvice`
- `HandlerInterceptor` — lives inside `DispatcherServlet`, sees the handler, three hooks; **Filters** wrap the whole servlet and know nothing about MVC

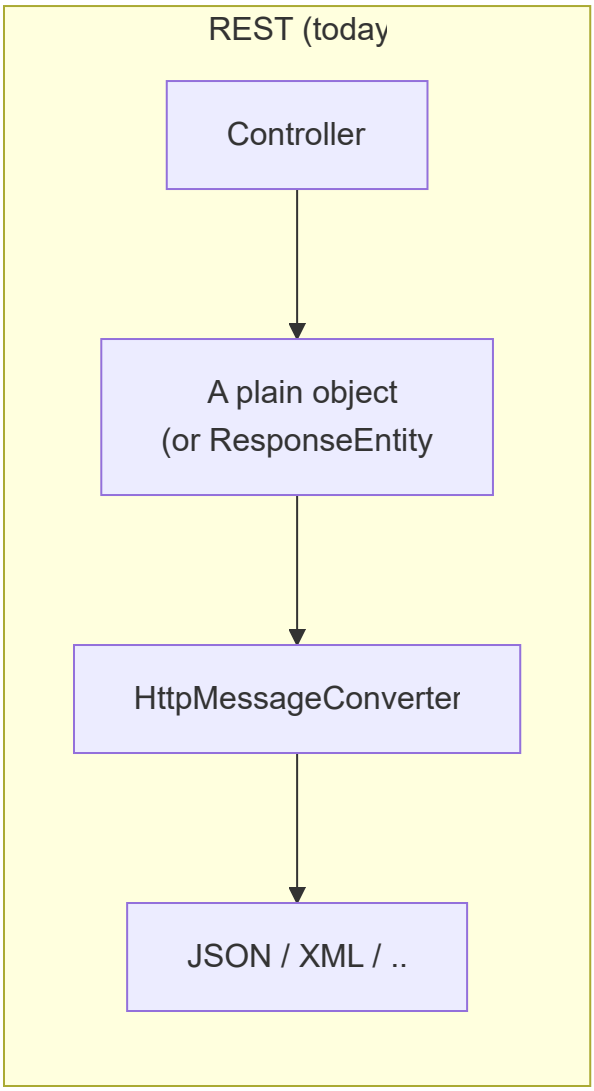
Today we don't relearn any of that. We change exactly one thing: what happens to the **return value** of a controller method.

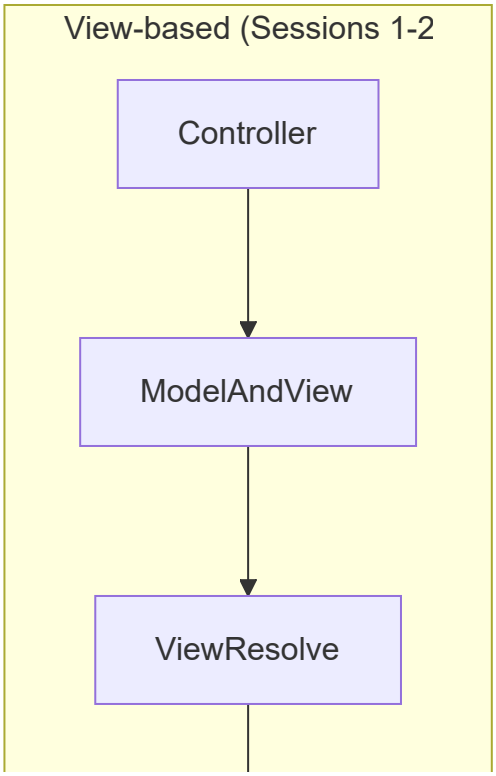
Session 3 Agenda

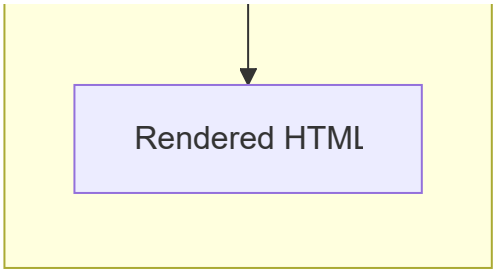
1. From MVC to REST — what actually changes
2. `@ResponseBody` and `@RestController`
3. HTTP message converters — and how `consumes` / `produces` really work
4. Designing REST endpoints — status codes, `ResponseEntity`
5. REST exception handling — JSON error bodies
6. `HandlerInterceptor` in a REST API — what's different, what isn't
7. The full picture — every component, one diagram
8. **Lab 3**

From MVC to REST

What Actually Changes


REST (today
Controller
A plain object
(or ResponseEntity
HttpMessageConverter
JSON / XML / ..


View-based (Sessions 1-2
Controller
ModelAndView
ViewResolve



- The **client** is different: a JS frontend, a mobile app, another service — something that wants **data**, not a rendered page
- Everything through `HandlerAdapter` is **identical** to Sessions 1–2
- The only thing that changes is the last step: **no** `ViewResolver`**, no** `View`**, no HTML** — the return value is converted straight into a response body

---

Side by Side

```
// Session 1-2 style
@Controller
public class ProductController {
    @GetMapping("/products/{id}")
    public String view(@PathVariable Long id, Model model) {
        model.addAttribute("product", repository.findById(id));
        return "products/view";              // a VIEW NAME
    }
}
```

```
// REST style
@RestController
public class ProductRestController {
    @GetMapping("/api/products/{id}")
    public Product view(@PathVariable Long id) {
        return repository.findById(id);       // the DATA itself
    }
}
```

Same `@GetMapping`, same `@PathVariable`, same `HandlerMapping` / `HandlerAdapter`. The method just returns something different, and gets annotated differently.

`@ResponseBody` and `@RestController`

`@ResponseBody` — Skip The View Entirely

```
@Controller
public class ProductController {

    @GetMapping("/api/products/{id}")
    @ResponseBody
    public Product view(@PathVariable Long id) {
        return repository.findById(id);
    }
}
```

`@ResponseBody` tells `RequestMappingHandlerAdapter`: *"don't treat this return value as a view name — hand it to a message converter and write it straight to the response body."* No `ViewResolver` is ever consulted for this method.

You can mix `@ResponseBody` methods into an otherwise view-based `@Controller` — some apps genuinely need a couple of JSON endpoints alongside HTML pages.

`@RestController` — The Shortcut

```
@RestController                 // = @Controller + @ResponseBody on every
method
@RequestMapping("/api/products")
public class ProductRestController {

    @GetMapping("/{id}")
    public Product view(@PathVariable Long id) {
        return repository.findById(id);
    }

    @GetMapping
    public List<Product> list() {
        return repository.findAll();
    }
}
```

`@RestController` is a **meta-annotation**: it applies `@ResponseBody` to *every* method in the class, so you never repeat it. If every method in a controller returns data, not a view, use `@RestController` from the start.

HTTP Message Converters

The Interface Doing The Actual Work

```
public interface HttpMessageConverter<T> {
    boolean canRead(Class<?> clazz, MediaType mediaType);
    boolean canWrite(Class<?> clazz, MediaType mediaType);
    T read(Class<? extends T> clazz, HttpInputMessage inputMessage);
    void write(T t, MediaType contentType, HttpOutputMessage outputMessage);
}
```

- `@RequestBody` → a converter's `read(...)` turns the incoming bytes into a Java object
- `@ResponseBody` / `@RestController` **return value** → a converter's `write(...)` turns the Java object into outgoing bytes

Same mechanism, both directions — just like `HandlerAdapter` was one interface with several implementations, so is this.

The Built-In Converters

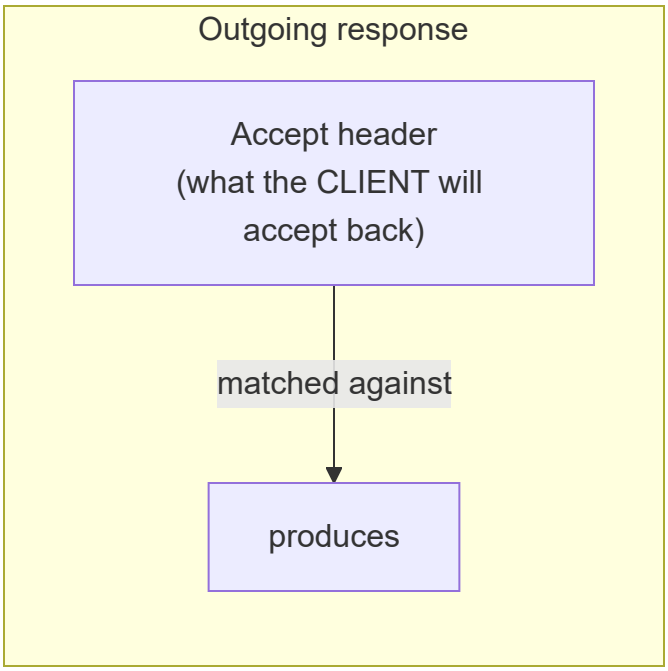
Converter	Handles
`MappingJackson2HttpMessageConverter`	JSON, via Jackson — **the one you'll use constantly**
`StringHttpMessageConverter`	plain `String` bodies
`FormHttpMessageConverter`	`application/x-www-form-urlencoded`
`ByteArrayHttpMessageConverter`	raw bytes
`Jaxb2RootElementHttpMessageConverter`	XML, if JAXB is present

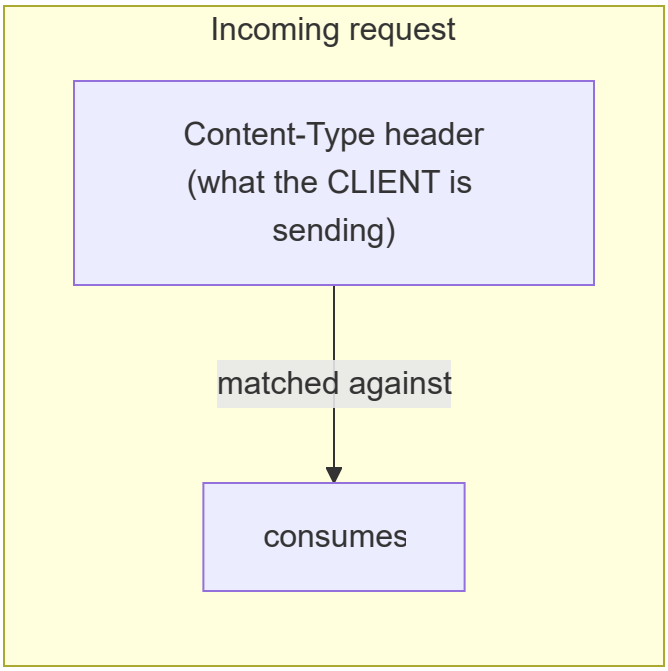
`@EnableWebMvc` registers a sensible default list automatically — **and specifically, it only registers the Jackson converter if `jackson-databind` is actually on the classpath.** Forget that dependency, and `@RestController` methods still run, but return an empty body or a `406` — another example of Session 1's "the defaults are bare minimums" idea, one dependency deep.

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.16.0</version>
</dependency>
```

`consumes` / `produces` — What They Actually Control

Two **separate** HTTP headers, two separate jobs:





Attribute	Matches against	Picks
`consumes`	request's `Content-Type`	which converter **reads** `@RequestBody`
`produces`	request's `Accept`	which converter **writes** the return value

They are independent — a client can `POST` JSON and ask to get XML back, or vice versa, as long as both are configured.

`consumes` in Action

```
@PostMapping(value = "/api/products", consumes = "application/json")
public Product createFromJson(@RequestBody Product product) {
    return repository.save(product);
}
```

- Client sends `Content-Type: application/json` → matches → Jackson's converter reads the body into a `Product`
- Client sends `Content-Type: text/xml` → **no match** → `415 Unsupported Media Type` — before your method even runs

Two methods can share a URL, split by what they accept in:

```
@PostMapping(value = "/api/products", consumes = "application/json")
public Product createFromJson(@RequestBody Product product) { ... }

@PostMapping(value = "/api/products", consumes = "application/x-www-form-
urlencoded")
public Product createFromForm(@ModelAttribute Product product) { ... }
```

`produces` in Action

```
@GetMapping(value = "/api/products/{id}", produces = "application/json")
public Product asJson(@PathVariable Long id) { return repository.findById(id);
}

@GetMapping(value = "/api/products/{id}", produces = "application/xml")
public Product asXml(@PathVariable Long id) { return repository.findById(id);
}
```

- Client sends `Accept: application/json` → first method matches
- Client sends `Accept: application/xml` → second method matches
- Client sends `Accept: text/csv` (and neither method produces that) → `406 Not Acceptable`

Memory aid: `consumes` is about what goes **in** — think "the server *consumes* your request body." `produces` is about what comes **out** — think "the server *produces* a response for you."

The Two Failure Codes, Side by Side

Status	Means	Caused by
415 Unsupported Media Type	"I don't know how to **read** what you sent"	`Content-Type` doesn't match any `consumes`
406 Not Acceptable	"I can't **write** a response in the format you asked for"	`Accept` doesn't match any `produces`

Both happen **before** your controller method body runs — `HandlerMapping` itself won't even select a handler that can't satisfy the negotiation.

Designing REST Endpoints

Resource-Oriented URLs

Bad (verb-oriented)	Good (resource-oriented)
`GET /getProduct?id=5`	`GET /products/5`
`POST /createProduct`	`POST /products`
`POST /deleteProduct?id=5`	`DELETE /products/5`
`GET /listAllActiveProducts`	`GET /products?active=true`

URLs name **things** (nouns); the HTTP **method** supplies the verb.

Method	Meaning	Idempotent?
`GET`	read	yes
`POST`	create	no
`PUT`	replace (full update)	yes
`PATCH`	partial update	no (usually)
`DELETE`	remove	yes

"Idempotent" = calling it 5 times has the same effect as calling it once — important for retries over a flaky network.

Status Codes That Actually Mean Something

Code	Use it for
`200 OK`	success, returning a body
`201 Created`	a `POST` that created something — include a `Location` header
`204 No Content`	success, nothing to return (e.g. a `DELETE`)
`400 Bad Request`	malformed input
`404 Not Found`	no such resource
`409 Conflict`	e.g. a duplicate unique key
`422 Unprocessable Entity`	well-formed but fails validation
`500 Internal Server Error`	your bug, not the client's

A REST API that returns `200` for everything (including errors, in the body) is throwing away half of HTTP for no reason.

`ResponseEntity<T>` — Full Control

Returning a plain object only ever gives you `200 OK`. For anything else, wrap it:

```
@PostMapping("/api/products")
public ResponseEntity<Product> create(@RequestBody Product product) {
    Product saved = repository.save(product);

    URI location = URI.create("/api/products/" + saved.getId());

    return ResponseEntity
            .created(location)      // 201 + Location header
            .body(saved);
}
```

```java
@GetMapping("/api/products/{id}")
public ResponseEntity<Product> view(@PathVariable Long id) {
    return repository.findById(id)
            .map(ResponseEntity::ok)                  // 200 + body
            .orElse(ResponseEntity.notFound().build()); // 404, no body
}
```

`ResponseEntity` carries **status + headers + body** — the REST equivalent of `ModelAndView` carrying **view name + model**.

REST Exception Handling

The Shape of an Error Response

REST clients need to **parse** errors, not read them off a rendered HTML page — so give errors a consistent shape:

```java
public record ErrorResponse(
        int status,
        String error,
        String message,
        String path,
        Instant timestamp
) {}
```

```json
{
  "status": 404,
  "error": "Not Found",
  "message": "No product with id 999",
  "path": "/api/products/999",
  "timestamp": "2025-01-01T10:15:30Z"
}
```

Every error your API returns should look like *some* predictable shape — callers will branch on `status` / `error` in code.

`@RestControllerAdvice` — The REST Version

```
@RestControllerAdvice                    // = @ControllerAdvice + @ResponseBody
everywhere
public class ApiExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ProductNotFoundException ex, HttpServletRequest request) {

        ErrorResponse body = new ErrorResponse(404, "Not Found",
                ex.getMessage(), request.getRequestURI(), Instant.now());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAny(Exception ex,
HttpServletRequest request) {
        ErrorResponse body = new ErrorResponse(500, "Internal Server Error",
                ex.getMessage(), request.getRequestURI(), Instant.now());
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(body);
    }
}
```

Exactly the same idea as Session 2's `@ControllerAdvice` — the difference is every handler returns `ResponseEntity<ErrorResponse>` (JSON), not a view name.

Validation Errors, REST-Style

```
@PostMapping("/api/products")
public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
    // no BindingResult parameter here -- if validation fails, Spring throws
    // MethodArgumentNotValidException BEFORE this method body ever runs
    return
ResponseEntity.status(HttpStatus.CREATED).body(repository.save(product));
}
```

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>>
handleValidation(MethodArgumentNotValidException ex) {
```

```
    Map<String, String> errors = new LinkedHashMap<>();
    ex.getBindingResult().getFieldErrors()
            .forEach(err -> errors.put(err.getField(),
err.getDefaultMessage()));
    return ResponseEntity.badRequest().body(errors);   // 400 + {"title":
"Title is required"}
}
```

Notice: **no `BindingResult` parameter** in `create(...)` this time. Without one to "catch" the errors, Spring throws instead — and your `@ExceptionHandler` catches that exception centrally, the same pattern as `ProductNotFoundException`.

`HandlerInterceptor` In A REST API

What's The Same

Everything from Session 2, unchanged:

- Still lives inside `DispatcherServlet`, still returned as part of the `HandlerExecutionChain` from `HandlerMapping`
- Still `preHandle` → controller → `postHandle` → (render) → `afterCompletion`
- Still the right tool for "this concern needs to know *which handler* is about to run" — vs. a Filter, which doesn't
- Registration is identical: `WebMvcConfigurer.addInterceptors(...)`

We are not re-teaching any of that today. Two things genuinely change.

Change #1 — `postHandle`'s `ModelAndView` Is `null`

```
@Override
public void postHandle(HttpServletRequest req, HttpServletResponse res,
                       Object handler, ModelAndView modelAndView) {
    // For a @RestController / @ResponseBody method, modelAndView is ALWAYS
null --
    // there never was a "model" or a "view name" to begin with, the return
    // value went straight to a message converter instead.
```

```
    if (modelAndView != null) {
        modelAndView.addObject("extra", "..."); // this branch never runs for
REST
    }
}
```

In Sessions 1–2, `postHandle` was a real place to enrich the model before rendering. For REST endpoints it's effectively **dead code** — the response body has often already been serialized by the time `postHandle` even runs. `preHandle` and `afterCompletion` remain fully useful; `postHandle` mostly isn't.

Change #2 — Short-Circuiting Must Return JSON

```
public class ApiKeyInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res,
Object handler)
            throws IOException {
        if (req.getHeader("X-Api-Key") == null) {
            res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            res.setContentType("application/json");
            res.getWriter().write("{\"error\":\"Missing X-Api-Key header\"}");
            return false;   // stop here -- controller never runs
        }
        return true;
    }
}
```

Session 2's `AuthInterceptor` wrote plain text for a browser to read. A REST client expects **JSON, even for errors** — so a blocked request has to look like every other error response your API returns, not a one-off string.

If You Need To Touch The Outgoing Body Itself

`postHandle` can't help (Change #1) — for REST APIs, Spring MVC has a purpose-built hook instead:

```java
@ControllerAdvice
public class WrapResponseAdvice implements ResponseBodyAdvice<Object> {

    @Override
    public boolean supports(MethodParameter returnType, Class converterType) {
        return true; // apply to every REST response
    }

    @Override
    public Object beforeBodyWrite(Object body, MethodParameter returnType,
            MediaType mediaType, Class converterType,
            ServerHttpRequest request, ServerHttpResponse response) {
        return Map.of("data", body, "timestamp", Instant.now()); // wrap every
response
    }
}
```

`ResponseBodyAdvice` runs **inside the message-converter step itself**, after your controller returns but before the bytes are written — the REST analog of "I want to touch the response on its way out," which is exactly what `postHandle` used to do for view-based apps.

Interceptor vs Filter vs AOP — One More Look

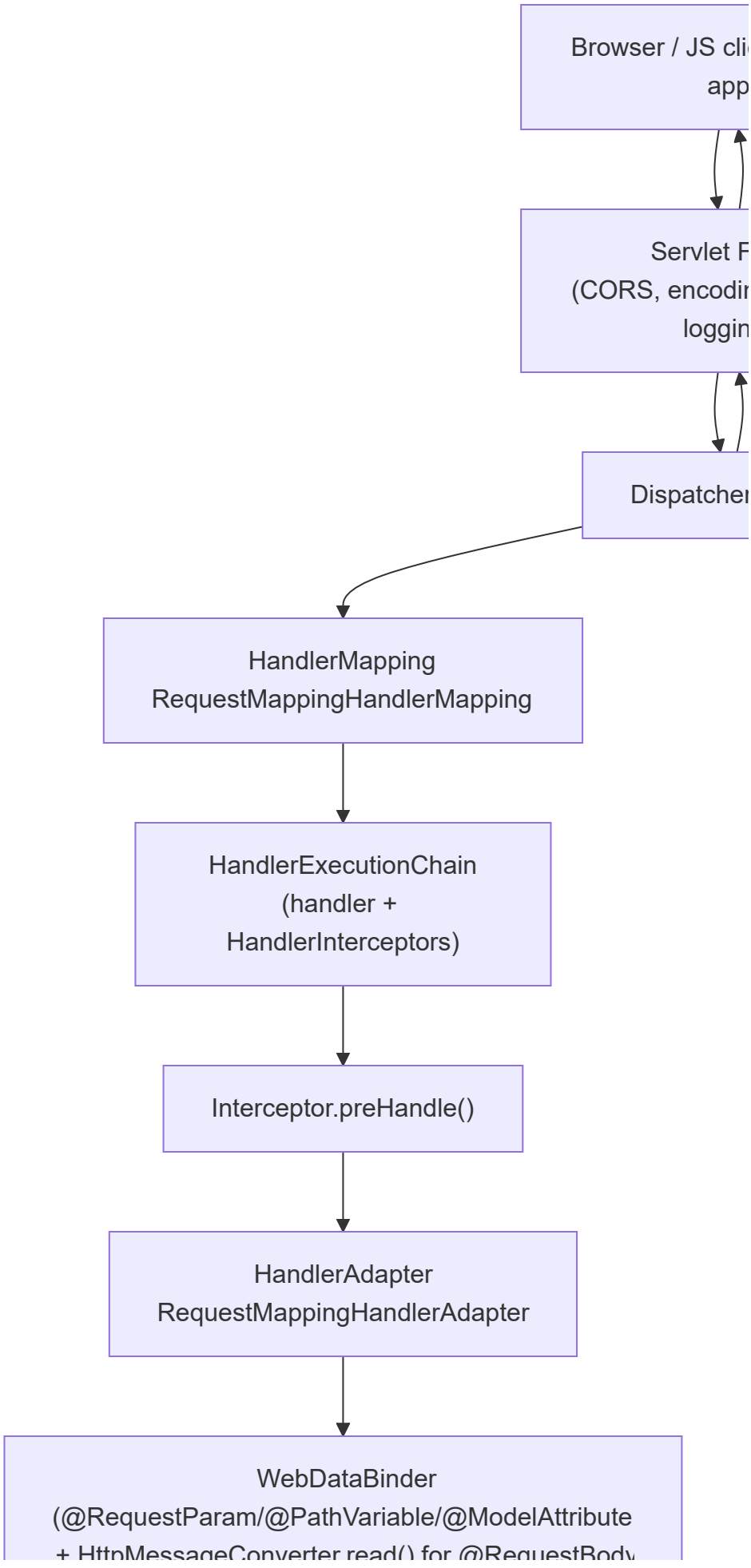
Same table as Session 2, still true — REST didn't change the layering, only *what a typical use case looks like* at the Interceptor layer:

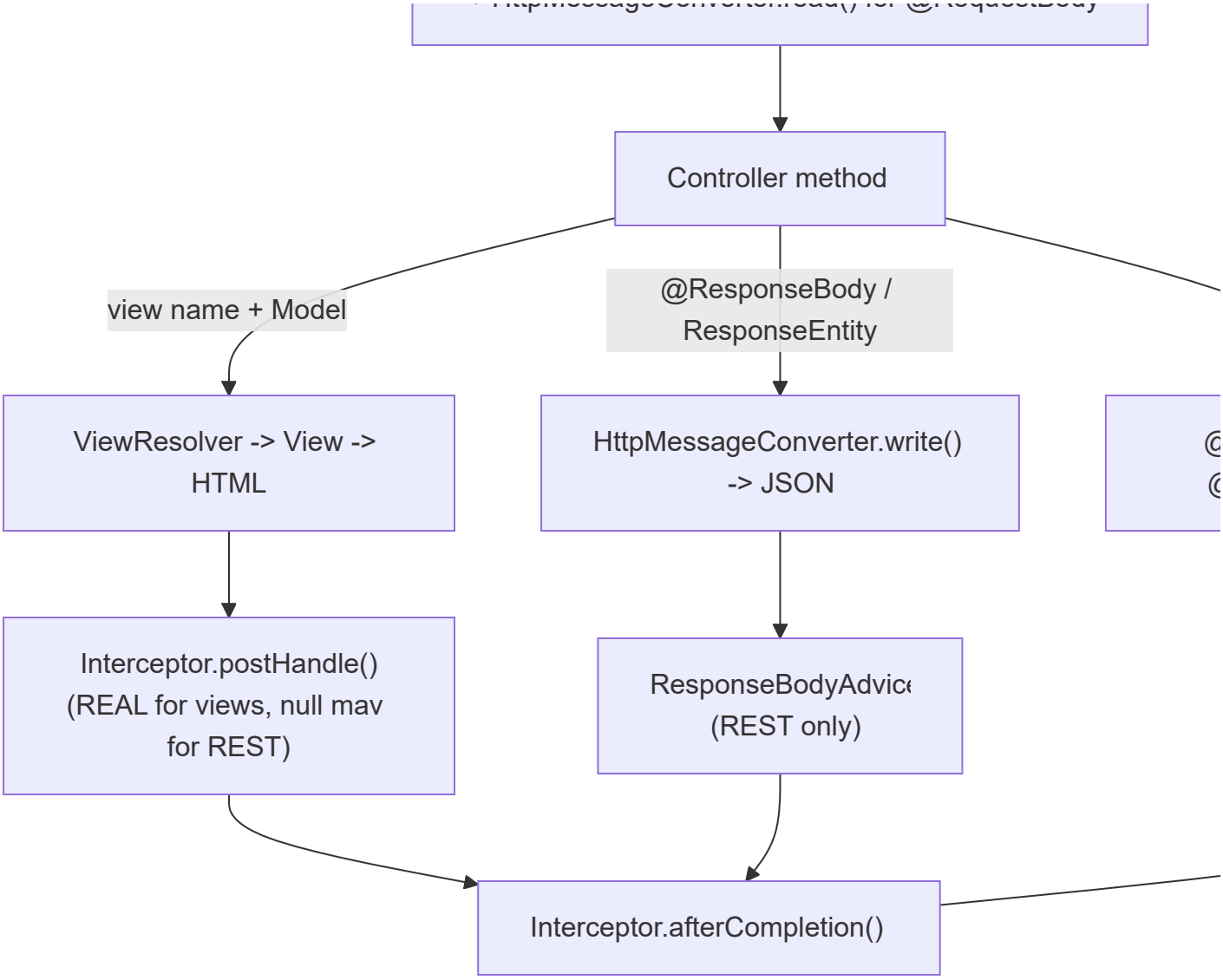
	Servlet Filter	**`HandlerInterceptor`**	**AOP (`@Aspect`)**
Sees	raw request/response	request/response + the handler	method, args, return value
Typical REST use	CORS headers, API gateway auth, compression	API-key checks, per-endpoint rate limiting, request logging	logging/transactions on the **service layer**, unrelated to HTTP at all

If a concern needs to run even when no controller matches at all (like CORS preflight on an unknown path) — that's still a Filter's job, not an interceptor's.

The Full Picture

Every Named Component, One Diagram


Browser / JS cli
app
Servlet F
(CORS, encodi
loggi
Dispatcher
HandlerMapping
RequestMappingHandlerMapping
HandlerExecutionChain
(handler +
HandlerInterceptors)
Interceptor.preHandle()
HandlerAdapter
RequestMappingHandlerAdapter
WebDataBinder
(@RequestParam/@PathVariable/@ModelAttribute
+ HttpMessageConverter.read() for @RequestBody



Three sessions, one pipeline — everything you've learned is one path through this same diagram.

Session 3 Recap

- REST changes the **last step only**: message converters instead of `ViewResolver` / `View`
- `@ResponseBody` / `@RestController` — skip view resolution, write the return value with a `HttpMessageConverter`
- `consumes` matches `Content-Type` (→ `415` if no match); `produces` matches `Accept` (→ `406` if no match)
- `ResponseEntity<T>` — status + headers + body, full control
- `@RestControllerAdvice` — the same exception-handling pattern from Session 2, returning JSON instead of a view

- `HandlerInterceptor` still works exactly as in Session 2 — just know `postHandle`'s model is dead for REST, and use `ResponseBodyAdvice` if you need to touch the outgoing body

---

Lab 3 — REST API For The Session 2 Entity

Setup: add `jackson-databind` to the Session 2 project (or a fresh copy).

Tasks

1. Build `ProductRestController` at `/api/products` — full CRUD: `GET` (list + by id), `POST` (`201` + `Location`), `PUT` (update, `200`), `DELETE` (`204`)
2. Use `ResponseEntity` throughout — correct status code for every branch, including a `404` for a missing id
3. Add `@Valid @RequestBody` to create/update; handle `MethodArgumentNotValidException` centrally, returning a field → message map
4. Build `@RestControllerAdvice` with a consistent `ErrorResponse` shape for `ProductNotFoundException` and a generic fallback
5. Add one endpoint with two `consumes` variants (JSON and form-urlencoded) and prove both work with `curl`
6. Build a small `ApiKeyInterceptor`, registered for `/api/**` only, that returns a JSON `401` when `X-Api-Key` is missing
7. **Bonus:** add a `ResponseBodyAdvice` that wraps every successful REST response in `{"data": ..., "timestamp": ...}`, and confirm it does **not** wrap your error responses (hint: check what `supports(...)` filters on)